KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Temple & Heritage Information Assistant

A Retrieval-Augmented Generation (RAG) System

Project ID: PRJ027

Team ID: T027

Team GitHub Repository: https://github.com/Yandrapalli-Pardha-Saradhi/project-

Student Name	Registration Number	Individual GitHub Link
Yandrapalli Pardha Saradhi	99230040457	https://github.com/Yandrapalli-Pardha-Saradhi/GEN-AI-Project
Rohith M	99230040751	https://github.com/rohith-m2006/GenAI
Bandameda Palli Jaya Prasad Reddy	99230040485	https://github.com/jayaprasad2005/Gen-AI-

Project Trainer / Guide

Shirwin Prince

Table of Contents

Section	Contents
Section 1	Introduction
Section 2	Problem Statement & Objectives
Section 3	System Architecture (RAG Pipeline)
Section 4	Implementation Details & Tech Stack
Section 5	Results and Testing
Section 6	Conclusion & Future Scope

1. Introduction

India has a rich collection of temples and heritage sites, each associated with distinctive history, architecture, traditions and cultural significance. Information about these places is often distributed across websites and reference materials, making it difficult for visitors and learners to locate relevant information quickly. The Temple & Heritage Information Assistant addresses this challenge through a conversational interface that helps users ask natural-language questions and receive answers grounded in a curated knowledge base, with source references to support further reading.

2. Problem Statement & Objectives

Problem: Users may need to search multiple sources to understand a temple's background, architectural features, traditions or other heritage information. General-purpose chatbots may provide unsupported or inaccurate details when they answer without consulting reliable, domain-specific material.

Objectives:

- To develop an AI assistant that answers natural-language questions about Indian temples and heritage.
- To organize curated information in a searchable vector database and retrieve relevant passages for each query.
- To use Retrieval-Augmented Generation (RAG) to ground generated answers in retrieved source content.
- To provide source-linked citations and avoid presenting unsupported information as verified fact.
- To deploy an accessible web application with a React frontend and FastAPI backend.

3. System Architecture (RAG Pipeline)

The application follows a Retrieval-Augmented Generation architecture. It has two main workflows: knowledge-base preparation (ingestion and indexing) and user query processing (retrieval and answer generation).

RAG Pipeline Diagram

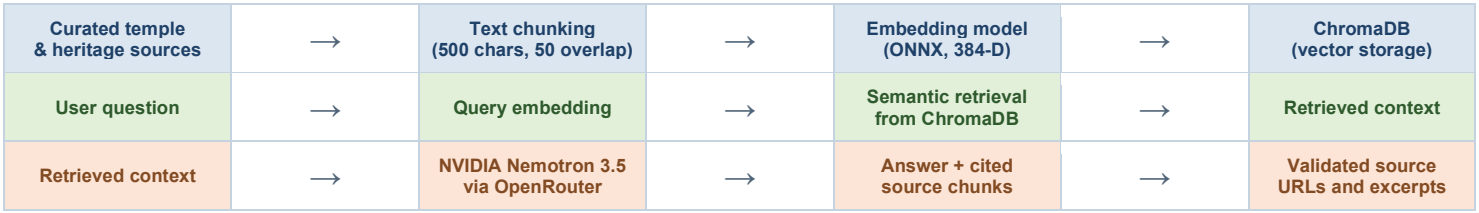


Figure 1. Knowledge ingestion, semantic retrieval, answer generation and citation validation.

3.1 Knowledge Preparation / Ingestion Phase

Curated temple and heritage source records are prepared with metadata such as source title and source URL. The content is divided into smaller, overlapping text chunks so that individual passages can be retrieved without losing nearby context. The embedding model converts each chunk into a numerical vector. These vectors, together with the text and metadata, are stored persistently in ChromaDB.

3.2 Query, Retrieval & Generation Phase

When a user submits a question, the backend converts it into an embedding using the same embedding model. ChromaDB performs semantic similarity search and returns relevant chunks. The backend supplies the retrieved passages as context to the configured Large Language Model through the OpenRouter-compatible API. The model generates a response based on that context. The backend validates references against the retrieved chunks and builds citations from stored source metadata, including source titles and URLs.

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend	React 18, TypeScript, Vite	User interface for asking questions and displaying answers and citations.
Backend	Python 3.10.11, FastAPI, Uvicorn	API endpoints, request validation and RAG workflow orchestration.
Vector Database	ChromaDB	Persistent storage and semantic retrieval of text embeddings.
Embedding Model	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	Converts multilingual text and queries into 384-dimensional vectors.
Embedding Runtime	ONNX Runtime, SentencePiece	Runs the embedding model in the optimized deployment configuration.
Large Language Model	NVIDIA Nemotron 3.5 Super VL via OpenRouter	Generates context-grounded natural-language answers.

4.1 Chunking and Retrieval Configuration

The backend uses sentence-aware text chunking with a default chunk size of approximately 500 characters and an overlap of 50 characters. The overlap helps preserve context across chunk boundaries. Each chunk is assigned a deterministic identifier and stored with its source metadata. The embedding representation has 384 dimensions.

```
Embedding model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Vector store: ChromaDB
Embedding dimension: 384
```

5. Results and Testing

The deployed application was checked through backend health verification, API connectivity and end-to-end chat queries. The live health endpoint reported an operational backend, an enabled generation configuration and the expected ChromaDB collection. Frontend and backend deployment connectivity, including the configured cross-origin access, was verified.

Functional testing included questions about the overview and traditions of Meenakshi Amman Temple. The system returned context-based answers with source citations for questions supported by the indexed knowledge base. A question about current temple timings returned an insufficient-evidence response because verified timing information was not available in the indexed sources. This behavior is intentional: the assistant should not invent details when the retrieved evidence is insufficient.

During response testing, a structured JSON-wrapped model response was identified as a frontend rendering edge case. Response normalization was added so structured answers are displayed as readable text while validated citations remain separate. Regression tests for this behavior passed.

Verification summary: 52 backend RAG tests passed in the recorded verification run; 5 frontend response-normalization regression tests passed; TypeScript type checking and the production frontend build completed successfully.

6. Conclusion & Future Scope

Conclusion: The Temple & Heritage Information Assistant demonstrates how RAG can make curated cultural and heritage information easier to explore through natural-language questions. Semantic retrieval with ChromaDB supplies relevant source passages, while NVIDIA Nemotron 3.5 generates responses using the retrieved context. Source-linked citations help users trace the information, and insufficient-evidence handling reduces the risk of unsupported answers.

Future Scope: Future improvements may include expanding and periodically reviewing the verified knowledge base, adding more reliable visitor information such as timings and facilities where authoritative sources are available, improving multilingual query handling, adding user feedback and retrieval evaluation, and introducing authentication or administrative tools for controlled content updates.